INF201 Deliverable 2: Campus Coffee Shop Sales Pipeline

GitHub repository: https://github.com/Sivert6/inf201-h26-deliverable-2

In [1]:
```python
import json
import re
from pathlib import Path

import numpy as np
import pandas as pd

DATA_DIR = Path("data")
```

Task 1: File ingestion and regular expressions

The log files use either UTF-8 or ISO-8859-1 encoding. Our function first tries UTF-8. If that fails, it uses ISO-8859-1 instead. ISO-8859-1 can read any file, so it will always work as a backup.

In [2]:
```python
def read_log_file(file_path: str | Path) -> tuple[str, str]:
    """Read a file. Try UTF-8 first, then ISO-8859-1."""
    path = Path(file_path)
    try:
        text = path.read_text(encoding="utf-8")
        return text, "utf-8"
    except UnicodeDecodeError:
        text = path.read_text(encoding="iso-8859-1")
        return text, "iso-8859-1"
```

In [3]:
```python
test_file = DATA_DIR / "2026" / "library_cafe" / "POS_01" / "register_sales.log"

text, encoding = read_log_file(test_file)
```

```
print("Encoding:", encoding)
print(text[:300])
```

```
Encoding: utf-8
[2026-10-15 07:00:14] [POS_01] item=Sandwich price=7.55 USD (SUCCESS)
[2026-10-15 07:00:50] [POS_01] item=Croissant price=4.26 EUR (CANCELLED)
[2026-10-15 07:02:34] [POS_01] item=Cappuccino price=4.59 USD (CANCELLED)
[2026-10-15 07:04:07] [POS_01] item=Espresso price=2.91 NOK (CANCELLED)
[2026-10-15
```

In [4]:
```
for register in ["POS_01", "POS_02"]:
    path = DATA_DIR / "2026" / "library_cafe" / register / "register_sales.log"
    _, enc = read_log_file(path)
    print(register, enc)
```

```
POS_01 utf-8
POS_02 utf-8
```

In [5]:
```
LOG_PATTERN = (
    r"\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] "
    r"\[(\w+)\] "
    r"item=(\w+) "
    r"price=(\d+\.\d+) "
    r"([A-Z]{3}) "
    r"\((SUCCESS|REFUND|CANCELLED)\)"
)
```

In [6]:
```
def parse_log(text: str) -> list[dict[str, str | float]]:
    """Find all valid sales lines and return them as a list of dictionaries."""
    records: list[dict[str, str | float]] = []
    for line in text.splitlines():
        match = re.match(LOG_PATTERN, line)
        if match:
            record = {
                "timestamp": match.group(1),
                "register_id": match.group(2),
                "item": match.group(3),
                "price": float(match.group(4)),
                "currency": match.group(5),
                "status": match.group(6),
            }
```

```
            records.append(record)
    return records
```

In [7]:
```
records = parse_log(text)

print("Number of valid records:", len(records))
print(records[0])
```

```
Number of valid records: 80
{'timestamp': '2026-10-15 07:00:14', 'register_id': 'POS_01', 'item': 'Sandwich', 'price': 7.55, 'currency': 'USD',
'status': 'SUCCESS'}
```

Discussion of Task 1

`read_log_file` first tries to read the file as UTF-8. If this gives a `UnicodeDecodeError`, it reads the file again as ISO-8859-1. This way the function finds the encoding by itself.

All log files are read as UTF-8, including those written with ISO-8859-1. The log lines only contain ASCII characters, which are encoded identically in both encodings. The ISO-8859-1 fallback is therefore never used with this data, but it would be needed if a file contained characters such as ø or €.

`parse_log` checks every line against a regular expression with six groups, one for each field. Lines that do not match, such as `--- PRINTER PAPER LOW ---`, are skipped. The test file gave 80 valid records, which matches the 80 sales lines the data generator writes per file.

Task 2: Recursive utility functions

Task 2.1: Recursive search for log files

The function walks through the folder structure using `Path.iterdir()`. If an item is a folder, the function calls itself on that folder. If it is a file ending in `.log`, it is added to the list.

In [8]:
```python
def find_log_files(directory: str | Path, extension: str = ".log") -> list[Path]:
    """Recursively find all files with the given extension.

    Uses explicit recursion with Path.iterdir(), not glob, rglob or os.walk.
    """
    directory = Path(directory)
    found_files: list[Path] = []

    for item in directory.iterdir():
        if item.is_dir():
            found_files.extend(find_log_files(item, extension))
        elif item.is_file() and item.suffix == extension:
            found_files.append(item)

    return found_files


log_files = find_log_files(DATA_DIR)
print(f"Found {len(log_files)} log files:")
for path in sorted(log_files):
    print(path)
```

```
Found 9 log files:
data/2026/library_cafe/POS_01/register_sales.log
data/2026/library_cafe/POS_02/register_sales.log
data/2026/library_cafe/POS_03/register_sales.log
data/2026/science_building/POS_01/register_sales.log
data/2026/science_building/POS_02/register_sales.log
data/2026/science_building/POS_03/register_sales.log
data/2026/student_union/POS_01/register_sales.log
data/2026/student_union/POS_02/register_sales.log
data/2026/student_union/POS_03/register_sales.log
```

Result: The function finds 9 log files, one for each of the three registers in all three stores. Note that `student_union` has log files, but is not listed in `store_config.json`. We return to this in Task 2.2.

Task 2.2: Recursive search for active registers

The function handles three cases. If the input is a dictionary, it checks whether `"active"` is `True` and stores the `"id"`, then searches all values. If the input is a list, it searches each element. Anything else ends the recursion and returns an empty set.

In [9]:
```python
def find_active_registers(data: object) -> set[str]:
    """Recursively search nested dicts and lists for registers
    where "active" is True, and return their ids.
    """
    active_ids: set[str] = set()

    if isinstance(data, dict):
        if data.get("active") is True and "id" in data:
            active_ids.add(data["id"])
        for value in data.values():
            active_ids |= find_active_registers(value)

    elif isinstance(data, list):
        for element in data:
            active_ids |= find_active_registers(element)

    return active_ids


with open(DATA_DIR / "store_config.json", encoding="utf-8") as f:
    config = json.load(f)

active_registers = find_active_registers(config)
print(active_registers)
```

```
{'POS_01', 'POS_03', 'POS_02'}
```

Result: `POS_04` is correctly excluded since it is inactive. However, the set only stores register IDs, not which store they belong to. Since `POS_01` to `POS_03` exist in all three stores in the log files, filtering on ID alone would also keep logs from `student_union`, which is not in the config at all.

Discussion of Task 2

Both functions are recursive with a clear base case and a recursive case. In `find_log_files`, a file is the base case and a folder triggers a new call. In `find_active_registers`, anything that is not a dictionary or a list ends the recursion. This means both

functions work for any depth of nesting, without knowing the structure in advance. `find_active_registers` does not depend on key names such as `"regions"` or `"stores"`, so it would still work if the config file were reorganised. It uses `is True`, so only real booleans count as active, not a string like `"true"`.

`Path.iterdir()` does not guarantee any particular order, so the file list is sorted before printing to make the output reproducible.

The config file and the log files do not match. The logs contain three stores with `POS_01` to `POS_03` in each, while the config only lists two stores, places `POS_03` in Science Building only, and includes `POS_04`, which has no log files. Store names are also written differently: `"Library Cafe"` in the config versus `library_cafe` in the folder names. Filtering on active register IDs alone is therefore not enough to identify which logs belong to active registers. A more precise approach would compare (store, register) pairs, which would require mapping the config names to the folder names.

Task 3: Functions for cleaning and analysing data using NumPy

Task 3.1: Removing price outliers

The function converts the prices to a 1D NumPy float array and computes the mean and standard deviation. A boolean mask marks every price that deviates more than 3 standard deviations from the mean. These prices are replaced with the median of the remaining valid prices.

In [10]:
```python
def remove_price_outliers(prices: list[float] | np.ndarray) -> np.ndarray:
    """Replace outliers with the median of the valid prices.

    A price is an outlier if it deviates from the mean by more
    than 3 standard deviations.
    """
    prices_array = np.array(prices, dtype=float).flatten()

    mean = prices_array.mean()
    std = prices_array.std()
    is_outlier = np.abs(prices_array - mean) > 3 * std

    median_valid = np.median(prices_array[~is_outlier])
```

```
    prices_array[is_outlier] = median_valid

    return prices_array


test_prices = [4.5, 3.0, 4.2, 3.5, 3.8, 7.5, 4.4, 2.9, 4.1, 3.6, 999.0, 4.6]
print(remove_price_outliers(test_prices))
```

```
[4.5 3.  4.2 3.5 3.8 7.5 4.4 2.9 4.1 3.6 4.1 4.6]
```

Result: The glitch value `999.0` is replaced with `4.1`, the median of the valid prices, while the legitimate high price `7.5` is kept. The 3σ rule needs enough data: with fewer than 11 values, no single value can deviate more than 3σ from the mean, which is why the test list has 12 values. The method also assumes outliers are rare. If more than about 10 % of the prices were glitches, the standard deviation would become so large that they would no longer be detected.

Task 3.2: Refund ratio

The function divides the refund count by the sales count element-wise. A small epsilon (10^{-6}) is added to the denominator so that division by zero never occurs.

In [11]:
```
def compute_refund_ratio(
    sales_count: np.ndarray, refund_count: np.ndarray
) -> np.ndarray:
    """Compute the refund ratio element-wise.

    A small epsilon is added to avoid division by zero.
    """
    epsilon = 1e-6
    return refund_count / (sales_count + epsilon)


sales = np.array([10, 0, 5])
refunds = np.array([2, 1, 0])
print(compute_refund_ratio(sales, refunds))
```

```
[1.9999998e-01 1.0000000e+06 0.0000000e+00]
```

Result: The ratios are approximately 0.2, 1 000 000 and 0. The first is slightly below 0.2 because of epsilon. The second shows the limitation of epsilon: it prevents a crash, but one refund with zero sales gives a meaningless ratio.

Discussion of Task 3

Both functions use vectorised NumPy operations on whole arrays instead of Python loops, which makes the code shorter and faster on large datasets. `remove_price_outliers` uses `flatten()`, which returns a copy, so the original price data is never modified. This avoids the problem where changing a slice of an array also changes the original, since NumPy slices are views.

The outlier detection uses one global mean, standard deviation and median for all items and currencies. This works here because the glitch value 999.00 is extreme compared to every normal price. It has two weaknesses. A smaller glitch, for example 30.00 for an espresso, would not be detected. In addition, an outlier is replaced with the median of all items, so a glitched sandwich gets the price of a typical item rather than a typical sandwich. Computing the median per item would give more accurate replacements.

The order of the cleaning steps in Task 5 also matters. Outliers are removed before NOK prices are multiplied by 10. If the currency correction came first, a NOK glitch would become 9990.00, and the corrected NOK prices would be ten times higher than the others. Both would inflate the standard deviation so much that the 999.00 glitches in EUR and USD could go undetected.

Task 4: Data analysis using Pandas

Task 4.1: Create the DataFrame

In [12]:
```python
def create_dataframe(records: list[dict]) -> pd.DataFrame:
    """Turn the list of sales into a DataFrame sorted by time."""
    df = pd.DataFrame(records)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp")
    df = df.sort_index()
    return df
```

```
df = create_dataframe(records)
df.head()
```

Out[12]:

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:14	POS_01	Sandwich	7.55	USD	SUCCESS
2026-10-15 07:00:50	POS_01	Croissant	4.26	EUR	CANCELLED
2026-10-15 07:02:34	POS_01	Cappuccino	4.59	USD	CANCELLED
2026-10-15 07:04:07	POS_01	Espresso	2.91	NOK	CANCELLED
2026-10-15 07:04:38	POS_01	Cappuccino	4.46	EUR	REFUND

Task 4.2: Hourly revenue per item

In [13]:

```
def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """Total price per item for each hour."""
    result = df.groupby("item").resample("1h")["price"].sum()
    result = result.reset_index()
    return result


hourly = hourly_revenue(df)
hourly.head(10)
```

Out[13]:

	item	timestamp	price
0	Cappuccino	2026-10-15 07:00:00	21.04
1	Cappuccino	2026-10-15 08:00:00	30.76
2	Cappuccino	2026-10-15 09:00:00	3.94
3	Croissant	2026-10-15 07:00:00	1033.61
4	Croissant	2026-10-15 08:00:00	11.14
5	Croissant	2026-10-15 09:00:00	7.47
6	Espresso	2026-10-15 07:00:00	12.24
7	Espresso	2026-10-15 08:00:00	24.48
8	Latte	2026-10-15 07:00:00	39.60
9	Latte	2026-10-15 08:00:00	1026.21

Task 4.3: Pivot table of revenue by item and status

In [14]:

```python
def status_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Total price per item, split into SUCCESS, REFUND and CANCELLED."""
    pivot = df.pivot_table(
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0,
    )
    return pivot


pivot = status_pivot_table(df)
pivot
```

Out[14]:

status	CANCELLED	REFUND	SUCCESS
item			
Cappuccino	4.59	22.27	28.88
Croissant	19.06	3.78	1029.38
Espresso	6.40	11.75	18.57
Latte	22.70	8.88	1039.04
Muffin	3.99	3.37	31.77
Sandwich	15.18	15.58	59.12

Task 4.4: Refund ratio per item

We count the number of SUCCESS and REFUND transactions for each item, and use `compute_refund_ratio` from Task 3 to calculate the ratio.

In [15]:

```
def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Refund ratio for each item, using compute_refund_ratio from Task 3."""
    counts = df.pivot_table(
        values="price",
        index="item",
        columns="status",
        aggfunc="count",
        fill_value=0,
    )

    sales_count = counts["SUCCESS"].to_numpy()
    refund_count = counts["REFUND"].to_numpy()
    ratio = compute_refund_ratio(sales_count, refund_count)

    result = pd.DataFrame({
        "item": counts.index,
        "sales_count": sales_count,
        "refund_count": refund_count,
        "refund_ratio": ratio,
```

```
    })
    return result


refund_ratios = refund_ratio_per_item(df)
refund_ratios
```

Out[15]:

	item	sales_count	refund_count	refund_ratio
0	Cappuccino	7	5	0.714286
1	Croissant	9	1	0.111111
2	Espresso	6	4	0.666667
3	Latte	10	2	0.200000
4	Muffin	9	1	0.111111
5	Sandwich	8	2	0.250000

Discussion of Task 4

`create_dataframe` turns the sales into a DataFrame sorted by time. `hourly_revenue` shows the total price per item for each hour, and `status_pivot_table` shows the total price per item for each status. `refund_ratio_per_item` uses `compute_refund_ratio` from Task 3, with the number of SUCCESS transactions as the sales count.

Here we only test on one log file with raw data, so the 999.00 glitches and the wrong NOK prices are still included. These are fixed in Task 5.

Task 5: Full workflow

We combine the functions from Task 1–4. The workflow runs on all log files, cleans the data and saves the four output files.

In [16]:
```
log_files = find_log_files(DATA_DIR)

with open(DATA_DIR / "store_config.json", encoding="utf-8") as f:
    config = json.load(f)
active_registers = find_active_registers(config)

all_records = []
for path in log_files:
    text, _ = read_log_file(path)
    for record in parse_log(text):
        if record["register_id"] in active_registers:
            all_records.append(record)

print("Log files:", len(log_files))
print("Active registers:", active_registers)
print("Records:", len(all_records))
```

```
Log files: 9
Active registers: {'POS_01', 'POS_03', 'POS_02'}
Records: 720
```

In [17]:
```
df = create_dataframe(all_records)
print("Highest price before cleaning:", df["price"].max())

df["price"] = remove_price_outliers(df["price"])
print("Highest price after cleaning:", df["price"].max())

is_nok = df["currency"] == "NOK"
df.loc[is_nok, "price"] = df.loc[is_nok, "price"] * 10
df["price"] = df["price"].round(2)

df.groupby("currency")["price"].mean()
```

```
Highest price before cleaning: 999.0
Highest price after cleaning: 7.99
```

Out[17]:
```
currency
EUR     4.247027
NOK    41.673384
USD     4.368426
Name: price, dtype: float64
```

In [18]:
```
hourly = hourly_revenue(df)
hourly["price"] = hourly["price"].round(2)

pivot = status_pivot_table(df).round(2)

refund_ratios = refund_ratio_per_item(df)
refund_ratios["refund_ratio"] = refund_ratios["refund_ratio"].round(4)

df.to_csv("cleaned_transactions.csv")
hourly.to_json("hourly_revenue.json", orient="records", date_format="iso")
pivot.to_csv("status_pivot_table.csv")
refund_ratios.to_csv("item_refund_ratios.csv", index=False)

print("All files saved.")
refund_ratios
```

```
All files saved.
```

Out[18]:

	item	sales_count	refund_count	refund_ratio
0	Cappuccino	77	24	0.3117
1	Croissant	83	22	0.2651
2	Espresso	86	18	0.2093
3	Latte	84	21	0.2500
4	Muffin	91	18	0.1978
5	Sandwich	70	12	0.1714

Discussion of Task 5

The workflow found 9 log files and 720 valid records. All registers in the log files (POS_01–POS_03) are active in the config, so the filter removed no records. This means that logs from `student_union` are included, even though this store is not listed in `store_config.json` (see Task 2.2).

The highest price was 999.00 before cleaning and below 10 after, so the glitches were removed. We remove outliers before fixing the NOK prices, as explained in Task 3. After the fix, NOK prices are about ten times higher than EUR and USD prices, as expected.

The totals in `hourly_revenue.json` and `status_pivot_table.csv` should be read with care. They add EUR, USD and NOK prices together without converting them to one currency, so the numbers mix currencies. The totals also include REFUND and CANCELLED transactions, which are not real income. A better report would convert all prices to one currency and only count SUCCESS transactions.